

TASKMANAGER

Backend Uygulaması Teknik Proje Dokümantasyonu

Mimari • API • Veritabanı • İş Kuralları • Test • Operasyon

Proje türü	ASP.NET Core Web API / Katmanlı Backend
Ana teknolojiler	.NET 10, EF Core 10, PostgreSQL, JWT, Docker, Serilog/Seq
Belge sürümü	1.0
Hazırlanma tarihi	10 Ağustos 2026
Yazar	<Ad Soyad>
Durum	Kod, migration ve testlerle doğrulanmış teknik referans

Bu belge mevcut kaynak kodu esas alır; parola, JWT anahtarı ve bağlantı bilgileri güvenli yer tutucularla gösterilir.

2. Revizyon Bilgisi

Sürüm	Tarih	Durum	Açıklama
1.0	10.08.2026	Yayın adayı	Güncel kaynak kod, 10 migration ve 131 test üzerinden yeniden oluşturuldu.

3. İçindekiler

2. Revizyon Bilgisi	2
3. İçindekiler	2
4. Yönetici Özeti	5
5. Proje Amacı ve Kapsamı	5
5.1 Amaç ve hedef kullanıcılar	5
5.2 Kapsama dahil	5
5.3 Kapsam dışı veya bulunmayan	5
6. Teknoloji Yığını	6
7. Solution ve Klasör Yapısı	6
8. Mimari	8
8.1 Bağımlılık yönü ve ayrım	8
8.2 Dependency Injection	8
8.3 Controller–service–repository akışı	8
9. İstek İşleme Akışı	9
10. Domain Modeli ve Veritabanı Tasarımı	10
10.1 Audit ve soft delete	11
10.2 Enum saklama	11
11. Temel Özellikler	11
11.1 Filtreleme, arama ve sıralama	12
12. Uzmanlık Tabanlı Görev Atama	13
12.1 UserRole ve UserExpertise ayrımı	13
12.2 Flags örneği	13
12.3 Etiketten göreve gereksinim	13
12.4 Atama ve eligible users	14
12.5 İlgili bileşenler	14
13. Authentication, Authorization ve Güvenlik	15
13.1 Registration ve login	15
13.2 JWT ve claim'ler	15
13.3 Yetki sınırları	15
13.4 Güvenlik sınırlamaları	16
14. Client ID Mekanizması	16
15. Validation ve İş Kuralları	16

15.1 İstek formatı ve iş kuralı ayrımı	16
16. Exception Handling	18
17. Logging ve Monitoring	18
18. API Referansı	19
AuthController	19
UsersController	19
ProjectsController	20
TasksController	20
TagsController	21
TaskTagsController	21
18.1 Önemli JSON örnekleri	22
19. DTO ve Mapping Genel Bakışı	22
20. Veri Erişimi ve Kalıcılık	23
20.1 Generic Repository	23
20.2 Unit of Work ve SaveChanges sahipliği	23
20.3 DbContext ve configuration	23
20.4 Repository sınırlamaları	23
21. Migrationlar ve Veritabanı Evrimi	24
22. Configuration	24
23. Yerel Kurulum ve Çalıştırma	26
23.1 Ön koşullar	26
23.2 Native çalıştırma	26
24. Docker ve Docker Compose	26
25. Kullanım Senaryoları	28
25.1 Uzmanlık kontrollü uçtan uca akış	28
25.2 Görev hiyerarşisi	28
25.3 Durum ve ilişki yönetimi	28
26. Test Stratejisi ve Sonuçları	28
27. Tasarım Kalıpları ve İlkeler	29
27.1 SOLID değerlendirmesi	29
28. Mevcut Sınırlamalar ve Bilinen Sorunlar	31
29. Gelecek Geliştirme Önerileri	31
29.1 Yüksek öncelik	31
29.2 Orta öncelik	32
29.3 Opsiyonel/ileri	32
30. Sorun Giderme	33
31. Sözlük	33
32. Nihai Proje Değerlendirmesi	35

32.1 Güçlü yönler 35

32.2 Dengeli olgunluk değerlendirmesi 35

32.3 Doğrulama durumu 35

Kullanım: Word'de belge açıldığında Ctrl+A, F9 ile içindekiler ve sayfa alanları yeniden güncellenebilir.

4. Yönetici Özeti

TaskManager; kullanıcı, proje, görev, alt görev, etiket ve çoklu kullanıcı atamalarını yöneten JWT korumalı bir ASP.NET Core Web API uygulamasıdır. Sistem, görevlerin proje ve parent-child bağlamında düzenlenmesini; etiketler üzerinden teknik uzmanlık gereksinimi tanımlanmasını ve yalnız uygun kullanıcıların atanmasını sağlar.

Çözüm Domain, Application, Infrastructure ve API katmanlarına ayrılmıştır. Controller'lar HTTP sözleşmesini, Application servisleri iş kurallarını, Infrastructure EF Core/PostgreSQL kalıcılığını, Domain ise bağımsız iş nesnelerini taşır. Generic Repository ve Unit of Work mevcut veri erişim yaklaşımıdır.

Olgunluk: Staj projesi düzeyinde güçlü bir katmanlama, merkezi doğrulama, audit/soft delete, JWT, yapılandırılmış loglama ve 131 başarılı otomatik test vardır. Üretim hazırlığı için güvenlik paketi güncellemesi, secret yönetimi, daha dar yetkilendirme, gerçek PostgreSQL entegrasyon testleri ve operasyonel dayanıklılık çalışmaları gerekir.

5. Proje Amacı ve Kapsamı

5.1 Amaç ve hedef kullanıcılar

Amaç; ekiplerin görevleri projeler altında oluşturması, öncelik ve durum takibi yapması, görevleri birden çok kullanıcıya ataması, alt görev ağacı kurması ve etiketlerle sınıflandırmasıdır. Hedef kullanıcılar sistem yöneticileri, yöneticiler ve görev üzerinde çalışan ekip üyeleridir; ancak mevcut API'de Manager rolüne özel bir yetki tanımı yoktur.

5.2 Kapsama dahil

- Kayıt, giriş, JWT üretimi ve Bearer doğrulaması.
- Kullanıcı, proje, görev ve etiket yönetimi; görev-etiket ilişkisi.
- Görev önceliği/durumu, proje bağlantısı ve parent-child hiyerarşisi.
- Bir görevin birden fazla kullanıcıya atanması ve duplicate atamanın engellenmesi.
- Authorization rolü ve teknik uzmanlığın ayrı yönetilmesi.
- Audit alanları, soft delete, global query filter, merkezi hata sözleşmesi ve Seq logları.

5.3 Kapsam dışı veya bulunmayan

- Proje üyeliği, kaynak sahipliği ve tenant izolasyonu.
- Atama kaldırma/transfer/tamamlama endpoint'leri; otomatik atama ve iş yükü dengeleme.
- Pagination, genel arama/sıralama, yorum, bildirim, rapor/dashboard ve task dependency.
- Refresh token, parola sıfırlama, rate limiting, hesap kilitleme ve API health endpoint'i.

6. Teknoloji Yığını

Alan	Teknoloji	Projede kullanım
Platform	.NET 10 / ASP.NET Core	API host, middleware ve controller katmanı.
ORM	Entity Framework Core 10.0.9	Model, migration, query filter ve PostgreSQL kalıcılığı.
Veritabanı	PostgreSQL / Npgsql 10.0.2	İlişkisel veri ve integer enum/flags saklama.
Validation	FluentValidation 12.1.1	DTO biçim ve sınır kuralları.
Mapping	AutoMapper 16.2.0	Entity ↔ DTO dönüşümleri.
Auth	JWT Bearer / HMAC-SHA256	Issuer, audience, lifetime ve signature doğrulaması.
Password	PasswordHasher<object>	Salt'lı parola hashleme ve doğrulama.
Logging	ILogger + Serilog + Seq	Yapılandırılmış uygulama/HTTP logları.
API docs	Swashbuckle 10.2.3	Swagger/OpenAPI ve Bearer güvenlik tanımı.
Test	xUnit 2.9.3, Moq 4.20.72, EF InMemory	Unit, controller, middleware ve persistence davranış testleri.
Container	Docker / Compose	API, postgres:18.4 ve Seq orkestrasyonu.

7. Solution ve Klasör Yapısı

```

TaskManager.slnx
├─ TaskManager.Domain/ Entity, enum, temel sınıflar
├─ TaskManager.Application/ DTO, validator, service, interface, mapping
├─ TaskManager.Infrastructure/ EF Core, repository, migration, güvenlik servisleri
├─ TaskManager.API/ Controller, middleware, Swagger, uygulama host'u
├─ TaskManager.Application.Tests/ xUnit/Moq/EF InMemory testleri
├─ TaskManager.API/Dockerfile
└─ docker-compose.yml

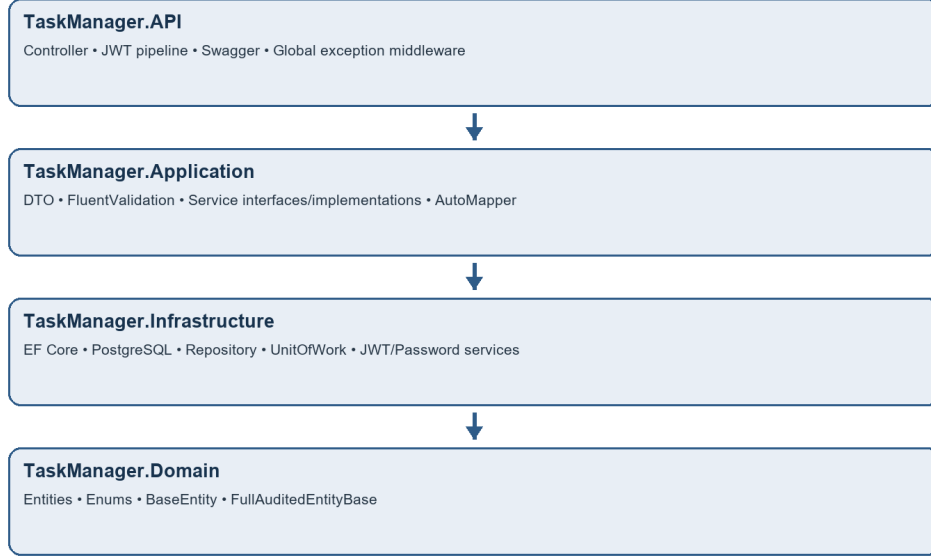
```

Proje	Bağımlılık	Sorumluluk
TaskManager.Domain	Başka proje referansı yok	Entity, enum ve audit temel sınıfları.
TaskManager.Application	Domain	Use-case servisleri, DTO, doğrulama, mapping ve portlar.
TaskManager.Infrastructure	Application + Domain	Portların EF Core/JWT/parola implementasyonları.
TaskManager.API	Application + Infrastructure	Composition root ve HTTP katmanı.
Application.Tests	API + Application + Infrastructure	Davranış ve sınır testleri.

Dosya adı notu: CreateUserRequest sınıfı CreatUserRequest.cs, UpdateUserRequest sınıfı UpdateUserRwquest.cs dosyasında bulunur. Sınıf adları doğru; dosya adlarında yazım hatası vardır.

8. Mimari

TaskManager Katmanlı Mimari



Şekil 1 — TaskManager katmanları ve sorumlulukları

8.1 Bağımlılık yönü ve ayırım

HTTP katmanı Application arayüzleri üzerinden servisleri kullanır. Application, kalıcılık ayrıntısını `IRepository<T>`, `IUnitOfWork` ve özel portlarla soyutlar. Infrastructure bu portları `AppDbContext`, `Repository<T>`, `UnitOfWork`, `JwtTokenService` ve `PasswordHasherService` ile gerçekleştirir. Domain iş modellerini framework bağımlılığından uzak tutar.

8.2 Dependency Injection

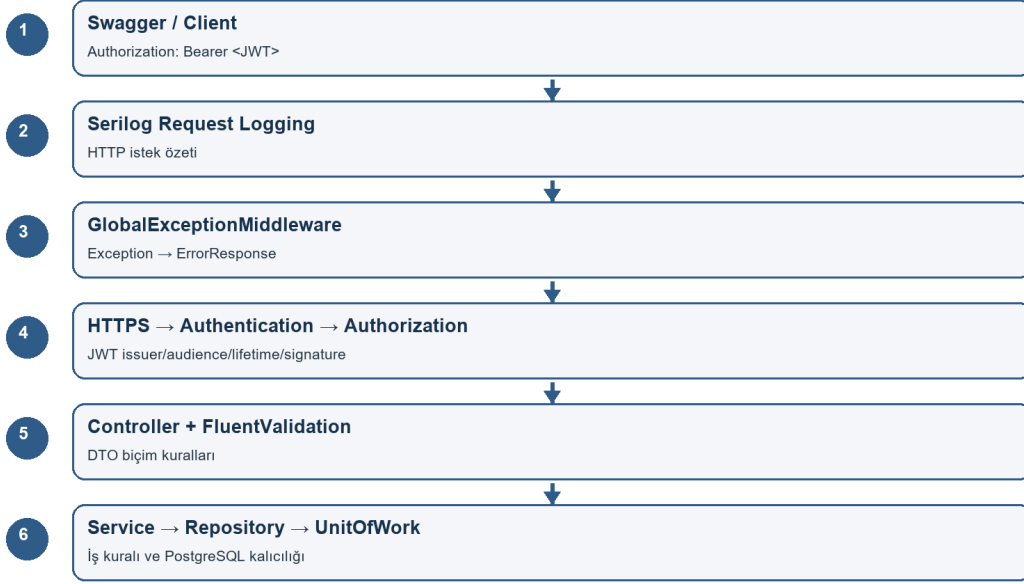
`Program.cs`; `AddApplication()` ve `AddInfrastructure(configuration)` uzantılarını çağırır. Application tüm validator, AutoMapper profil ve use-case servislerini scoped kaydeder. Infrastructure `AppDbContext`, generic repository, özel `TaskDetailRepository`, `UnitOfWork` ve güvenlik servislerini scoped kaydeder.

8.3 Controller–service–repository akışı

Controller yalnız HTTP parametrelerini alır, FluentValidation sonucunu `BaseApiController` yardımcılarıyla 400'e dönüştürür ve servise delegasyon yapar. Servis varlık kontrolü, duplicate, hiyerarşi ve uzmanlık gibi iş kurallarını uygular. Repository EF Core `DbSet` üzerinde sorgu/değişiklik yapar; `SaveChangesAsync` çağırısı `UnitOfWork` üzerinden tek noktadan gerçekleştirilir.

9. İstek İşleme Akışı

HTTP İstek Yaşam Döngüsü



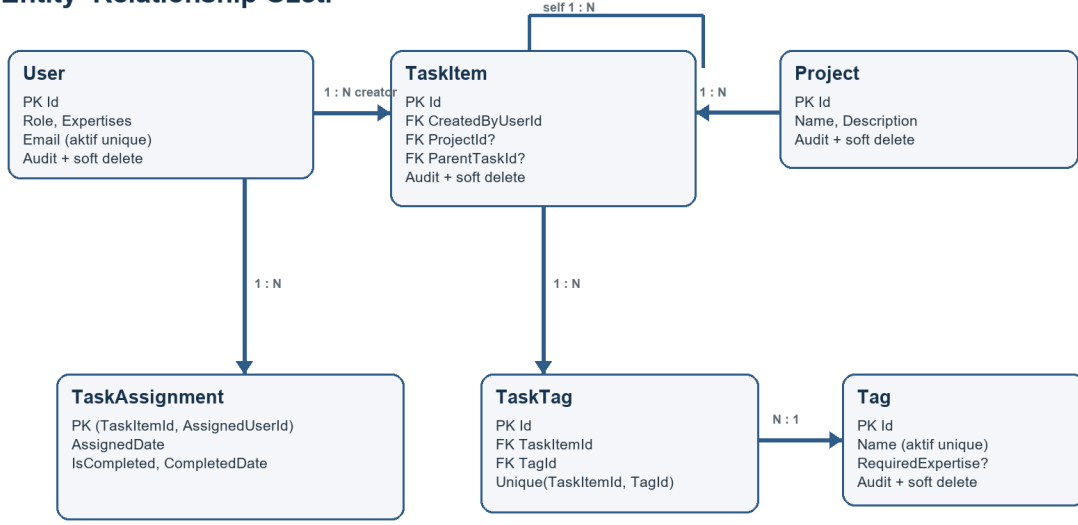
Şekil 2 — Program.cs ile doğrulanan HTTP istek akışı

1. UseSwagger ve UseSwaggerUI tüm ortamlarda etkinleşir.
2. UseSerilogRequestLogging HTTP isteğini yapılandırılmış olarak kaydeder.
3. GlobalExceptionHandler aşağı katmanlardan gelen exception'ları yakalar.
4. UseHttpsRedirection çalışır; ardından UseAuthentication JWT'yi doğrular.
5. UseAuthorization controller/action üzerindeki [Authorize] kurallarını uygular.
6. MapControllers route'u çözer; controller FluentValidation çalıştırır.
7. Service iş kurallarını, repository sorgularını ve UnitOfWork kaydını yürütür.
8. Başarılı sonuç DTO olarak; beklenen hata ErrorResponse olarak döner.

Güncel durum: Program.cs içinde ClientContextMiddleware bulunmaz. X-Client-Id istek akışının parçası değildir.

10. Domain Modeli ve Veritabanı Tasarımı

Entity–Relationship Özeti



Şekil 3 — EF Core konfigürasyonlarından türetilen ilişki özeti

Entity	PK	FK / önemli alan	Navigation	Kısıt / yaşam döngüsü
User	Id	Email; Role; Expertises	CreatedTasks; TaskAssignments	Full audit + soft delete; aktif email unique
TaskItem	Id	CreatedByUserId; ProjectId?; ParentTaskId?	Creator; Project; Parent/Subtasks; Assignments; Tags	Full audit + soft delete; Restrict ilişkiler
Project	Id	—	Tasks	Full audit + soft delete
Tag	Id	RequiredExpertise?	TaskTags	Full audit + soft delete; aktif name unique
TaskTag	Id	TaskItemId; TagId	TaskItem; Tag	BaseEntity; fiziksel delete; pair unique
TaskAssignment	TaskItemId + AssignedUserId	İki FK	TaskItem; AssignedUser	Composite PK; navigation query filter

10.1 Audit ve soft delete

FullAuditedEntityBase; CreatorId, LastModifiedDate, LastModifierId, IsDeleted, DeletedDate ve DeleterId alanlarını taşır. AppDbContext.SaveChanges/SaveChangesAsync, Added/Modified/Deleted state'lerinde bu alanları merkezi günceller. Delete state'i audited entity için Modified'a çevrilir ve satır fiziksel olarak kaldırılmaz.

User, Project, TaskItem ve Tag doğrudan !IsDeleted query filter'ına sahiptir. TaskAssignment ve TaskTag, bağlı user/task/tag silinmişse navigation tabanlı filter ile gizlenir. TaskTag FullAuditedEntityBase değildir; ilişki endpoint'i fiziksel silme yapar.

10.2 Enum saklama

UserRole, TaskItemStatus, TaskPriority ve UserExpertise PostgreSQL integer sütunlarında tutulur. UserExpertise [Flags] olduğu için birden fazla değer tek integer içinde bit toplamı olarak saklanır. Tag.RequiredExpertise nullable integer'dır ve validator tek bir flag ile sınırlar.

11. Temel Özellikler

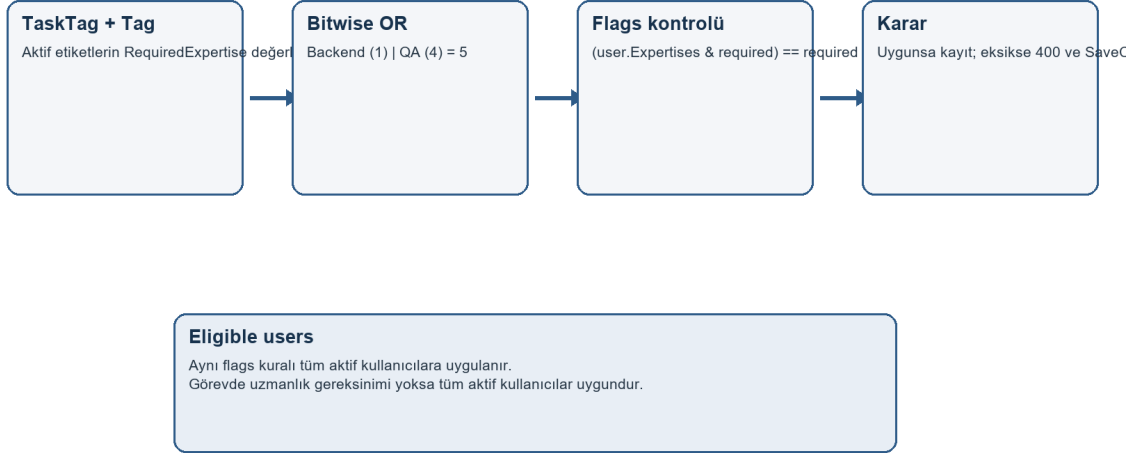
Özellik	Ana sınıflar	Davranış	Kritik kural/not
Kullanıcı yönetimi	UsersController, UserService	Listeleme/görüntüleme/oluşturma/güncelleme/s oft delete; Admin rol ve uzmanlık güncellemesi.	Aktif e-posta unique; CreateUserRequest parola içermez.
Görev yönetimi	TasksController, TaskService	CRUD, priority, due date, status ve ayrıntılı görünüm.	Create sırasında status Pending; creator JWT'den.
Proje yönetimi	ProjectsController, ProjectService	Listeleme, görüntüleme, oluşturma, güncelleme ve proje görevleri.	Service delete var; controller DELETE route yok.
Görev hiyerarşisi	TaskHierarchyService	Parent-child ağaç, project uyumu ve cycle algılama.	Parent/project yalnız create isteğinde belirlenir.
Etiket yönetimi	TagsController, TagService	CRUD, ad normalizasyonu ve expertise gereksinimi.	Ad trim + lowercase; aktif isim unique.
Görev-etiket	TaskTagsController, TaskTagService	Toplu bağlama, filtreleme, ilişki değiştirme/silme.	TaskTag.Id ile işlem; duplicate pair engellenir.
Çoklu atama	TaskAssignmentService	Bir göreve birden çok kullanıcı; kullanıcı başına composite PK.	Aynı user-task tekrarına 409.
Kimlik/yetki	AuthService, JwtTokenService	Register/login, role claim ve [Authorize].	Çoğu işlem herhangi authenticated kullanıcıya açık.
Log ve hata	ILogger, Serilog, Seq, middleware	Yapılandırılmış operasyon logları ve standart hata gövdesi.	Yalnız beklenmeyen 500 middleware'de error loglanır.

11.1 Filtreleme, arama ve sıralama

Genel pagination, search veya client-selectable sorting uygulanmamıştır. Yalnız GET /api/tasktags endpoint'i taskId ve tagId query filtrelerini destekler. TaskDetailRepository tagleri ada, atamaları AssignedDate'e göre iç sıralar.

12. Uzmanlık Tabanlı Görev Atama

Uzmanlık Tabanlı Atama



Şekil 4 — Uzmanlık gereksiniminin hesaplanması ve karar akışı

12.1 UserRole ve UserExpertise ayrımı

UserRole (User=1, Admin=2, Manager=3) kullanıcının ne yapmaya yetkili olduğunu belirler. UserExpertise (None=0, Backend=1, Frontend=2, QA=4, DevOps=8) teknik yeterliliği ifade eder. İki kavram ayrı entity alanı, ayrı endpoint ve ayrı iş kuralı olarak tutulur; expertise JWT role claim'ine eklenmez.

12.2 Flags örneği

```

Backend = 1 // 0001
QA = 4 // 0100
Backend | QA = 5 // 0101

(user.Expertises & requiredExpertises) == requiredExpertises
  
```

Örneğin görev Backend+QA (5) ister ve kullanıcı Backend+QA+DevOps (13) taşırsa 13 & 5 sonucu 5 olur; eşitlik sağlandığı için fazla uzmanlık atamayı engellemez.

12.3 Etiketten göreve gereksinim

Tag.RequiredExpertise null, None veya tek bir tanımlı uzmanlık olabilir. Bir tag'de birleşik flags reddedilir; çoklu gereksinim birden fazla tag ile ifade edilir. TaskAssignmentService aktif TaskTag ve Tag kayıtlarını sorgular, nullable değerleri OR ile birleştirir. Etiket adı hiçbir kararda kullanılmaz.

12.4 Atama ve eligible users

AssignTaskAsync sırasıyla görev, hedef kullanıcı, duplicate atama ve uzmanlık uygunluğunu kontrol eder. Eksik uzmanlık varsa isimleri içeren BadRequestException fırlatır; AddAsync ve SaveChangesAsync çağrılmaz. GET /api/tasks/{taskId}/eligible-users aynı matcher ile aktif kullanıcıları filtreler, atama oluşturmaz ve otomatik seçim yapmaz. requiredExpertises=None ise bütün aktif kullanıcılar eşleşir.

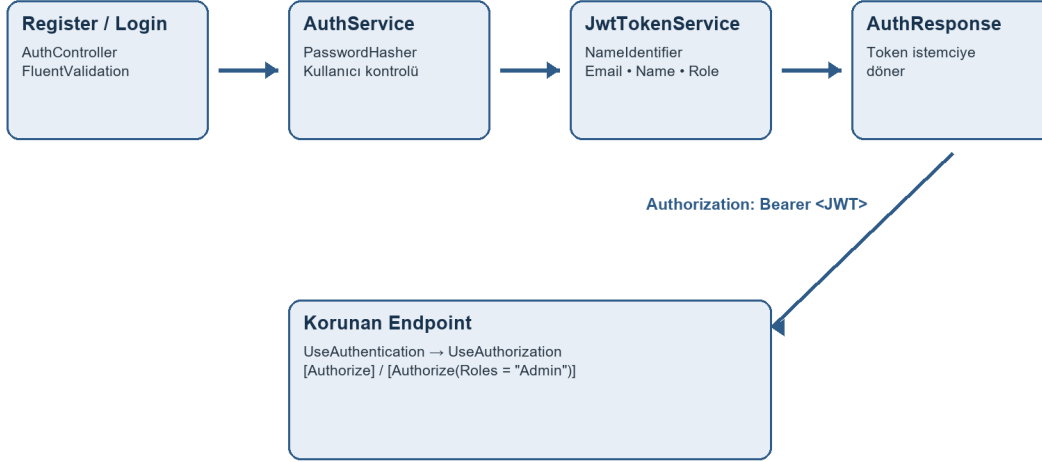
12.5 İlgili bileşenler

- Domain: UserExpertise.cs, User.Expertises, Tag.RequiredExpertise.

- DTO/validator: UpdateUserExpertisesRequest, EligibleUserResponse, TagRequest, UpdateUserExpertisesRequestValidator, TagRequestValidator.
- Service/API: UserService.UpdateUserExpertisesAsync, TaskAssignmentService, UsersController ve TasksController.
- Persistence: UserConfiguration, TagConfiguration ve 20260810112620_AddExpertiseBasedTaskAssignment.
- Test: ExpertiseAssignmentServiceTests, TagExpertiseTests, UserServiceTests ve ValidatorBoundaryTests.

13. Authentication, Authorization ve Güvenlik

JWT Kimlik Doğrulama Akışı



Şekil 5 — JWT üretim ve yetkilendirme akışı

13.1 Registration ve login

RegisterRequest; firstName, lastName, email ve password alır. AuthService aktif e-posta tekrarını 409 ile engeller, PasswordHasher<object> ile hash üretir, user kaydeder ve JWT içeren AuthResponse döndürür. Login aynı genel hata mesajıyla bulunmayan kullanıcıyı ve yanlış parolayı 401'e dönüştürür.

13.2 JWT ve claim'ler

JwtTokenService; ClaimTypes.NameIdentifier, Email, Name ve Role claim'lerini üretir; HmacSha256 kullanır. Token süresi yapılandırmada 60 dakikadır. Program.cs issuer, audience, lifetime ve signing key doğrulamalarını açar. Role güncellendikten sonra eski token eski role claim'ini taşımaya devam eder; yeniden login gerekir.

13.3 Yetki sınırları

- AuthController [AllowAnonymous].
- Diğer bütün controller'lar sınıf seviyesinde [Authorize].
- Yalnız GET /api/users, PATCH /api/users/{id}/role ve PATCH /api/users/{id}/expertises Admin rolü ister.
- Manager rolüne özel action/policy yoktur.
- Kaynak sahipliği kontrolü yoktur; herhangi authenticated kullanıcı user/task/tag/project üzerinde birçok mutation yapabilir.

13.4 Güvenlik sınırlamaları

- System.Security.Cryptography.Xml 8.0.3 için build sırasında beş yüksek önem dereceli NU1903 advisory'si oluşur.
- appsettings.json içinde literal JWT key bulunduğu doğrulanmıştır; bu belgede gösterilmez ve <JWT_SECRET> ile dışarı alınmalıdır.
- Compose API servisi Jwt_Key environment mapping'i tanımlamaz; secret override gerekir.
- Swagger bütün ortamlarda açıktır; rate limiting, refresh/revocation, lockout ve health endpoint'i yoktur.
- Başarılı/başarısız auth logları e-posta içerir; PII saklama politikası gerekir.

14. Client ID Mekanizması

Projede bulunamadı: Güncel kaynakta X-Client-Id header'ı, ClientContextMiddleware, ClientIdHeaderOperationFilter veya HttpContext.Items tabanlı Client ID saklama davranışı yoktur. Program.cs bu middleware'i kaydetmez; Swagger yalnız Bearer JWT tanımlar.

Bu nedenle Client ID ile JWT arasında güncel bir ayırım/iş birliği davranışı anlatılamaz. Audit aktörü ICurrentUserService aracılığıyla JWT içindeki NameIdentifier claim'inden alınır. Eski dokümanlarda Client ID geçiyorsa tarihsel bilgi olarak değerlendirilmelidir; mevcut API isteğinde X-Client-Id gönderilmesi gerekmez ve eksikliğine özel bir response yoktur.

15. Validation ve İş Kuralları

15.1 İstek formatı ve iş kuralı ayırımı

FluentValidation; boş alan, uzunluk, e-posta biçimi, enum tanımı, pozitif ID, due date ve liste tekrarları gibi veritabanından bağımsız kuralları uygular. Servisler ise kaydın varlığı, aktif e-posta/tag adı, duplicate assignment/TaskTag, parent-project uyumu ve expertise yeterliliği gibi veri bağımlı kuralları yürütür.

Kural	Katman	Uygulayan sınıf
Ad/e-posta/title sınırları	Validator	İlgili *RequestValidator
DueDate bugün veya gelecek	Validator	CreateTaskRequestValidator / UpdateTaskRequestValidator
ProjectId/ParentTaskId/AssignedUserId > 0	Validator	Task validator'ları
Aktif e-posta unique	Service + DB index	AuthService/UserService + UserConfiguration
Tag adı trim + lowercase ve aktif unique	Service + DB index	TagService + TagConfiguration
TaskIds boş değil, unique, >0	Validator	TagTasksRequestValidator
TaskTag pair unique	Service + DB index	TaskTagService + TaskTagConfiguration
Görev ile parent aynı proje	Service	TaskHierarchyService
Cycle algılama	Service (tree read)	TaskHierarchyService.BuildTaskTree
Duplicate assignment	Service + composite PK	TaskAssignmentService + configuration
Tag tek expertise	Validator	TagRequestValidator
User expertise list unique/defined	Validator	UpdateUserExpertisesRequestValidator
Atanan kullanıcı tüm gereksinimleri taşır	Service	TaskAssignmentService

Tutarlılık notu: ValidationMessages çoğu mesajı merkezileştirir; UpdateUserRoleRequestValidator mesajı doğrudan string olarak tanımlar.

16. Exception Handling

Exception	HTTP	Kullanım
BadRequestException	400	Geçersiz iş kuralı veya ID/uzmanlık durumu
UnauthorizedException	401	Yanlış login veya geçersiz creator claim
NotFoundException	404	Aktif entity bulunamadı
ConflictException	409	Duplicate veya hierarchy cycle
Diğer Exception	500	Güvenli genel mesaj; ayrıntı response'a çıkmaz

AppException HttpStatusCode taşır. GlobalExceptionHandler beklenen AppException türlerini kendi mesajlarıyla; beklenmeyen exception'ları 'An unexpected error occurred.' mesajıyla döndürür. Yalnız 500 durumları LogError ile exception ayrıntısı içerir.

```
{
  "statusCode": 404,
  "message": "Task was not found.",
  "path": "/api/tasks/999/eligible-users",
  "traceId": "<TRACE_ID>"
}
```

FluentValidation hataları ErrorResponse biçiminde değildir. BaseApiController bunları field/message nesnelerinden oluşan bir JSON array ile 400 olarak döndürür.

```
[
  { "field": "AssignedUserId", "message": "AssignedUserId must be greater than 0." }
]
```

17. Logging ve Monitoring

Servisler ILogger<T> ile success, warning ve önemli kimlikleri message-template biçiminde yazar. Program.cs Serilog'u appsettings üzerinden kurar, request logging middleware'ini kullanır ve Seq sink'ine gönderir. Docker Compose içinde Seq http://localhost:5341 adresine yayınlanır; API container'ı sink URL'sini http://seq:80 olarak override eder.

- Structured property örnekleri: UserId, TaskId, ProjectId, TagId, AssignedUserId, RequiredExpertises.
- Global middleware yalnız beklenmeyen 500 hatalarını exception stack ile error seviyesinde loglar.
- Client ID log alanı yoktur; audit/log korelasyonu JWT user ID ve TraceId üzerinden yapılır.
- Seq veri kalıcılığı external taskmanager-seq-data volume'ündedir.

18. API Referansı

Toplam 33 route controller attribute'larından çıkarılmıştır. JSON enum değerleri global string converter olmadığı için sayısal gönderilir. Controller sınıfındaki [Authorize] aksi belirtilmedikçe bütün action'lara uygulanır.

AuthController

Method	Route	Auth/rol	İstek	Yanıt	Başarı	Hata / not
POST	/api/auth/register	Anonymous	RegisterRequest	AuthResponse	200	400, 409. Benzersiz aktif e-posta; parola hashlenir; JWT döner.
POST	/api/auth/login	Anonymous	LoginRequest	AuthResponse	200	400, 401. Aktif kullanıcı ve doğru parola gerekir.

UsersController

Method	Route	Auth/rol	İstek	Yanıt	Başarı	Hata / not
GET	/api/users	JWT/Admin	—	UserResponse[]	200	401, 403. Global filter nedeniyle yalnız aktif kullanıcılar.
GET	/api/users/{id}	JWT	—	UserResponse	200	401, 404. Route: id. Kayıt yoksa controller 404.
POST	/api/users	JWT	CreateUserRequest	UserResponse	200	400, 401, 409. Parola almaz; atama hedefi kullanıcı kaydı üretir.
PUT	/api/users/{id}	JWT	UpdateUserRequest	UserResponse	200	400, 401, 404, 409. Ad, soyad ve e-posta güncellenir.
DELETE	/api/users/{id}	JWT	—	—	204	401, 404. Soft delete; audit alanları merkezi doldurulur.
PATCH	/api/users/{id}/role	JWT/Admin	UpdateUserRoleRequest	UserResponse	200	400, 401, 403, 404. Authorization rolünü günceller.
PATCH	/api/users/{id}/expertises	JWT/Admin	UpdateUserExpertisesRequest	UserResponse	200	400, 401, 403, 404. Liste flags değerine dönüştürülür.

ProjectsController

Method	Route	Auth/rol	İstek	Yanıt	Başarı	Hata / not
GET	/api/projects	JWT	—	ProjectResponse[]	200	401. Aktif projeleri listeler.
GET	/api/projects/{id}	JWT	—	ProjectResponse	200	401, 404. ProjectService NotFoundException üretir.
GET	/api/projects/{id}/tasks	JWT	—	TaskResponse[]	200	401, 404. Projeye bağlı aktif görevler.
POST	/api/projects	JWT	CreateProjectRequest	ProjectResponse	200	400, 401. Yeni proje ve create audit.
PUT	/api/projects/{id}	JWT	UpdateProjectRequest	Boş gövde	200	400, 401, 404. İmza ProjectResponse olsa da controller Ok() döndürür.

TasksController

Method	Route	Auth/rol	İstek	Yanıt	Başarı	Hata / not
GET	/api/tasks	JWT	—	TaskResponse[]	200	401. Aktif görevleri listeler.
GET	/api/tasks/{id}	JWT	—	TaskDetailResponse	200	401, 404. Project, creator, tag ve assignment detayları.
GET	/api/tasks/{id}/tree	JWT	—	TaskTreeResponse	200	401, 404, 409. İstenen düğümden başlayan görev ağacı; cycle kontrolü.
POST	/api/tasks	JWT	CreateTaskRequest	TaskResponse	200	400, 401, 404. Creator JWT claim'inden; status zorunlu Pending.
PUT	/api/tasks/{id}	JWT	UpdateTaskRequest	TaskResponse	200	400, 401, 404. Title, description, priority, due date; creator korunur.
DELETE	/api/tasks/{id}	JWT	—	—	204	401, 404. Soft delete.
POST	/api/tasks/{taskId}/assign	JWT	AssignTaskRequest	TaskResponse	200	400, 401, 404, 409. Duplicate ve uzmanlık uygunluğu kontrol edilir.
GET	/api/tasks/{taskId}/eligible-users	JWT	—	EligibleUserResponse[]	200	401, 404. Tüm gereksinimleri karşılayan aktif kullanıcılar.
PUT	/api/tasks/{taskId}/status	JWT	UpdateTaskStatusRequest	TaskResponse	200	400, 401, 404. Tanımlı TaskItemStatus değeri gerekir.

TagsController

Method	Route	Auth/rol	İstek	Yanıt	Başarı	Hata / not
GET	/api/tags	JWT	—	TagResponse[]	200	401. Aktif etiketler.
GET	/api/tags/{id}	JWT	—	TagResponse	200	401, 404. RequiredExpertise dahil.
POST	/api/tags	JWT	TagRequest	TagResponse	200	400, 401, 409. Ad trim + lowercase; tek expertise veya null.
PUT	/api/tags/{id}	JWT	TagRequest	—	204	400, 401, 404, 409. Ad ve RequiredExpertise güncellenir.
DELETE	/api/tags/{id}	JWT	—	—	204	401, 404. Tag soft delete; ilişkiler query filter ile gizlenir.

TaskTagsController

Method	Route	Auth/rol	İstek	Yanıt	Başarı	Hata / not
GET	/api/tasktags?taskId=&tagId=	JWT	Query: taskId?, tagId?	TaskTagResponse[]	200	401. Tek mevcut genel filtreleme endpoint'i.
POST	/api/tasktags/tags/{tagId}/tasks	JWT	TagTasksRequest	—	204	400, 401, 404. Yeni ilişkileri ekler; mevcut ilişkileri atlar.
DELETE	/api/tasktags/{taskId}	JWT	—	—	204	401, 404. TaskTag BaseEntity olduğundan fiziksel silme.
GET	/api/tasktags/{id}	JWT	—	TaskTagDetailResponse	200	401, 404. TaskTag.Id kullanılır; Tag.Id değildir.
PUT	/api/tasktags/{id}	JWT	UpdateTaskTagRequest	TaskTagDetailResponse	200	400, 401, 404, 409. İlişkideki tag değiştirilir; duplicate engellenir.

18.1 Önemli JSON örnekleri

```
PATCH /api/users/12/expertises
{ "expertises": [1, 4] }
```

```
POST /api/tags
{ "name": " backend ", "requiredExpertise": 1 }
```

```
POST /api/tasks/25/assign
{ "assignedUserId": 12 }
```

```
GET /api/tasks/25/eligible-users
[
  { "id": 12, "firstName": "Demo", "lastName": "User", "expertises": 5 }
]
```

19. DTO ve Mapping Genel Bakışı

DTO katmanı entity'lerin veri tabanı/navigation ayrıntılarını HTTP sözleşmesinden ayırır. PasswordHash hiçbir response DTO'sunda bulunmaz. Request/response ayrımı over-posting riskini azaltır; örneğin CreateTaskRequest CreatedByUserId içermez ve creator JWT'den gelir.

Grup	Request DTO	Response DTO	Not
Auth	RegisterRequest, LoginRequest	AuthResponse	Token döner; password dönmez.
Users	Create/Update/Role/Expertises	UserResponse, EligibleUserResponse	UserResponse Role içermez; Eligible email içermez.
Tasks	Create/Update/Status/Assign	TaskResponse, Detail, Tree, Assignment	Detail özel repository ile manuel map edilir.
Projects	CreateProjectRequest, UpdateProjectRequest	ProjectResponse	Basit AutoMapper eşlemesi.
Tags	TagRequest, TagTasksRequest, UpdateTaskTagRequest	TagResponse, TaskTag*Response	TaskTag DTO'ları servis içinde manuel oluşturulur.

MappingProfile MemberList.Source ile create/update kaynak alanlarının karşılandığını test edilebilir kılar. TaskDetailRepository, ilişkili veriyi Include/ThenInclude ile alır ve projection'ı manuel yapar. Mevcut manuel mapping TagResponse.RequiredExpertise alanını TaskDetailResponse.Tags içinde doldurmaz; detay endpoint'inde bu alan null görünür.

20. Veri Erişimi ve Kalıcılık

20.1 Generic Repository

Repository<T>; GetAllAsync, GetByIdAsync(params object[]), FindAsync, AddAsync, Update ve Delete sağlar. DbSet üzerinde çalışır ve global query filter'ları doğal olarak uygular. GetByIdAsync ayrıca FullAuditedEntityBase.IsDeleted=true ise null döndürür.

20.2 Unit of Work ve SaveChanges sahipliği

UnitOfWork yalnız ApplicationDbContext.SaveChangesAsync'i sarmalar. Servisler bir use-case içindeki repository değişikliklerini hazırladıktan sonra save çağırır. Açık transaction, retry policy veya concurrency token uygulanmamıştır; tek SaveChanges kendi EF transaction davranışına dayanır.

20.3 DbContext ve configuration

ApplicationDbContext altı DbSet tanımlar ve ApplyConfigurationsFromAssembly ile ayrı EntityTypeConfiguration sınıflarını yükler. PostgreSQL provider DependencyInjection.AddInfrastructure içinde UseNpgsql ile seçilir. İlişkiler çoğunlukla DeleteBehavior.Restrict kullanır; aggregate soft delete edildiğinde join satırları korunur ve query filter ile görünmez.

20.4 Repository sınırlamaları

- Pagination/projection/include kompozisyonu generic arayüzde yoktur; GetAllAsync tam liste materialize eder.
- CancellationToken ve AsNoTracking seçenekleri generic metotlarda sunulmaz.
- TaskDetail için özel repository gerekli olmuştur; eligible users tüm aktif kullanıcıları belleğe alıp filtreler.

21. Migrationlar ve Veritabanı Evrimi

Zaman	Migration	Şema etkisi
20260703102300	InitialCreate	Users, Tasks, TaskAssignments; ilk FK/index yapısı.
20260710112249	AddPasswordHashToUser	Users.PasswordHash varchar(500), mevcut satırlar için boş default.
20260716104041	AddRoleToUser	Users.Role integer, default User=1.
20260720090235	AddPriorityToTask	Tasks.Priority integer, default Medium=2.
20260720205006	AddProjectsAndTaskHierarchy	Projects; Tasks.ProjectId ve ParentTaskId nullable FK.
20260721223451	AddTaskTagRelationship	Tags ve ilk composite-key TaskTags tablosu.
20260722205514	AddFullAuditFields	User/Task/Tag/Project için audit ve soft-delete sütunları.
20260728083736	AddTaskTagPrimaryKey	TaskTags.Id/CreatedDate; pair için unique index.
20260809211903	CompleteSoftDeleteBehavior	Aktif kayıt partial unique indexleri ve join FK'lerde Restrict.
20260810112620	AddExpertiseBasedTaskAssignment	Users.Expertises default 0; Tags.RequiredExpertise nullable.

```
dotnet ef migrations list --project TaskManager.Infrastructure/TaskManager.Infrastructure.csproj
--startup-project TaskManager.API/TaskManager.API.csproj --context AppDbContext

dotnet ef database update --project TaskManager.Infrastructure/TaskManager.Infrastructure.csproj
--startup-project TaskManager.API/TaskManager.API.csproj --context AppDbContext
```

10 Ağustos 2026 doğrulamasında EF modeli son migration ile uyumludur: 'No changes have been made to the model since the last migration.' Uzmanlık migration'ı daha önce yerel PostgreSQL'e uygulanmış ve veri sayılarının korunduğu doğrulanmıştır.

22. Configuration

Anahtar	Güvenli örnek	Kaynak
ConnectionStrings:DefaultConnection	<CONNECTION_STRING>	appsettings, user-secrets veya environment
Jwt:Key	<JWT_SECRET>	Environment/user-secrets; repository'ye yazılmamalı
Jwt:Issuer	TaskManagerApi	appsettings
Jwt:Audience	TaskManagerClient	appsettings
Jwt:ExpirationMinutes	60	appsettings
Serilog.WriteTo: Seq	http://localhost:5341 / container http://seq:80	appsettings + compose override
ASPNETCORE_ENVIRONMENT	Development (Compose)	environment
POSTGRES_*	<DB_USER>, <DB_PASSWORD>, TaskManagerDb	.env / deployment secret

Secret güvenliği: Mevcut literal Jwt:Key değeri belgelenmemiştir. Development için dotnet user-secrets; Docker/CI için secret environment veya güvenli secret store kullanılmalıdır.

23. Yerel Kurulum ve Çalıştırma

23.1 Ön koşullar

- .NET 10 SDK
- Docker Desktop veya erişilebilir PostgreSQL
- dotnet-ef global/local tool
- İsteğe bağlı DBeaver/psql

23.2 Native çalıştırma

1. ConnectionStrings__DefaultConnection ve Jwt__Key değerlerini güvenli şekilde sağlayın.
2. Bağımlılıkları restore edin ve çözümü build edin.
3. Migration'ları hedef development veritabanına uygulayın.
4. API projesini çalıştırın ve Swagger'ı açın.
5. Register/login yanıtındaki token'ı Swagger Authorize alanına girin; 'Bearer' yazmadan token değeri yeterlidir.

```
dotnet restore TaskManager.slnx
dotnet build TaskManager.slnx
dotnet ef database update --project TaskManager.Infrastructure/TaskManager.Infrastructure.csproj
--startup-project TaskManager.API/TaskManager.API.csproj
dotnet run --project TaskManager.API/TaskManager.API.csproj
```

- HTTP Swagger: http://localhost:5177/swagger
- HTTPS Swagger: https://localhost:7285/swagger
- X-Client-Id gönderilmez; yalnız JWT gerekir.

24. Docker ve Docker Compose

Dockerfile iki aşamalıdır: mcr.microsoft.com/dotnet/sdk:10.0 image'ında restore/publish; mcr.microsoft.com/dotnet/aspnet:10.0 image'ında minimal runtime. API 8080 portunu expose eder ve dotnet TaskManager.API.dll ile başlar.

Servis	Image	Port	Volume	Bağımlılık/not
postgres	postgres:18.4	5432	postgres-data	pg_isready healthcheck
seq	datalust/seq:latest	5341→80	taskmanager-seq-data (external)	ACCEPT_EULA=Y
api	taskmanager-api:latest	8080	—	postgres healthy + seq started

```
docker volume create taskmanager-seq-data
docker compose config --quiet
docker compose up -d --build
docker compose ps
docker compose logs -f api
docker compose down
```

Compose tanımı 10 Ağustos 2026'da docker compose config --quiet ile geçerli bulundu. Son dokümantasyon kontrolü anında servisler çalışmıyordu; bu nedenle o anda canlı Swagger/container health testi yapılmadı.

25. Kullanım Senaryoları

25.1 Uzmanlık kontrollü uçtan uca akış

1. POST /api/auth/register ve /login ile JWT alın.
2. Admin token ile PATCH /api/users/{id}/expertises gövdesinde [1,4] göndererek Backend+QA atayın.
3. POST /api/projects ile proje oluşturun.
4. POST /api/tasks ile projectId kullanarak Pending görev oluşturun.
5. POST /api/tags ile requiredExpertise=1 ve ikinci tag için requiredExpertise=4 oluşturun.
6. POST /api/tasktags/tags/{tagId}/tasks ile iki tag'i göreve bağlayın.
7. GET /api/tasks/{taskId}/eligible-users ile Backend+QA kullanıcılarını listeleyin.
8. POST /api/tasks/{taskId}/assign ile uygun kullanıcıyı atayın; 200 bekleyin.
9. Yalnız Backend kullanıcılarını atamayı deneyin; eksik QA mesajlı 400 ve yeni assignment olmaması beklenir.

25.2 Görev hiyerarşisi

1. Bir proje ve ana görev oluşturun.
2. Aynı projectId ve parentTaskId ile alt görev oluşturun.
3. GET /api/tasks/{parentId}/tree ile nested Subtasks listesini doğrulayın.
4. Farklı projectId ile aynı parent altında görev oluşturmaya çalışın; 400 bekleyin.

25.3 Durum ve ilişki yönetimi

1. PUT /api/tasks/{taskId}/status ile status=2 (InProgress) gönderin.
2. GET /api/tasktags?taskId={taskId} ile TaskTag.Id değerini bulun.
3. PUT /api/tasktags/{taskTagId} ile başka Tag.Id değerine taşıyın.
4. DELETE /api/tasktags/{taskTagId} ile ilişkiyi fiziksel silin.

26. Test Stratejisi ve Sonuçları

Tek test projesi TaskManager.Application.Tests net10.0 hedefler. xUnit test runner, Moq ile interface izolasyonu ve EF Core InMemory ile persistence/integration-benzeri testler kullanılır. Gerçek PostgreSQL container'ına karşı otomatik entegrasyon paketi yoktur.

Alan	Başlıca kapsam
Controller	Validation short-circuit, authenticated creator claim, 200/204 sözleşmeleri
Service	Auth, user, project, task, tag, hierarchy, assignment ve TaskTag iş kuralları
Expertise	Flags uygunluğu, eksik uzmanlıkta persistence olmaması, eligible users
Validator	Enum sınırları, ID, duplicate listeler, None kombinasyonu, due date
Persistence	Audit, soft delete, query filter, hard-delete join davranışı
Middleware/security	ErrorResponse, safe 500, JWT NamelIdentifier claim, CurrentUserService
Mapping	AutoMapper configuration validity

```
dotnet test TaskManager.slnx --no-restore --no-build
```

Doğrulanmış sonuç: 131 başarılı, 0 başarısız, 0 atlanan; toplam 131 test. Coverage collector paketi vardır fakat bu doğrulamada coverage yüzdesi üretilmemiştir.

27. Tasarım Kalıpları ve İlkeler

Kalıp	Kanıt	Değerlendirme
Repository	<code>IRepository<T>/Repository<T></code>	EF Core veri erişimini Application portu arkasına alır.
Unit of Work	<code>IUnitOfWork/UnitOfWork</code>	Use-case sonunda tek <code>SaveChangesAsync</code> noktası.
Service Layer	*Service sınıfları	Controller'dan iş kurallarını ayırır.
DTO	Application/DTOs	Entity ve hassas alanları HTTP sözleşmesinden ayırır.
Dependency Injection	Program + <code>AddApplication/AddInfrastructure</code>	Implementasyonların composition root'ta bağlanması.
Middleware	<code>GlobalExceptionHandler</code>	Cross-cutting hata sözleşmesi.
Options	<code>IOptions<JwtSettings></code>	JWT yapılandırmasının typed tüketimi.
Composite benzeri	TaskItem parent/Subtasks + recursive tree	Hiyerarşik görev ağacını aynı tipte temsil eder.

27.1 SOLID değerlendirmesi

- SRP büyük ölçüde iyi: validation, business, persistence ve HTTP ayrı sınıflardadır.
- DIP iyi: Application servisleri repository/UnitOfWork/security arayüzlerine bağımlıdır.
- OCP kısmi: generic repository tekrarları azaltır; ancak yeni zengin sorgular için özel repository gerekir.
- ISP uygun ölçüde küçük arayüzlerle desteklenir; IUserService/ITaskService büyümeye açıktır.
- LSP açısından belirgin bir ihlal gözlenmemiştir.
- Bazı controller response sözleşmeleri ve manuel mapping alanları tutarsızdır; API contract disiplini zayıftır.

28. Mevcut Sınırlamalar ve Bilinen Sorunlar

Öncelik	Alan	Kanıt/etki
Yüksek	NU1903 güvenlik advisory'leri	System.Security.Cryptography.Xml 8.0.3 için beş yüksek severity uyarı.
Yüksek	Secret yönetimi	Literal JWT key ve Compose'ta Jwt__Key mapping eksikliği.
Yüksek	Yetkilendirme sınırı	Admin dışı authenticated kullanıcılar birçok mutation yapabilir; ownership yok.
Orta	UserResponse rolü	Role update 200 döner fakat UserResponse Role alanı içermez.
Orta	Task detail expertise	TaskDetailRepository TagResponse.RequiredExpertise alanını map etmez.
Orta	User creation	POST /api/users parola almaz; bu kayıt doğrudan login hesabı olarak kullanılamaz.
Orta	Project delete	ProjectService.DeleteAsync var; controller route yok.
Orta	Assignment lifecycle	Remove/transfer/complete endpoint'i yok; IsCompleted alanı kullanılmıyor.
Orta	Liste ölçeklenmesi	Pagination/search/sorting yok; eligible users bellekte filtrelenir.
Orta	Concurrency/transaction	Concurrency token, açık transaction ve retry stratejisi yok.
Orta	Integration testing	Gerçek PostgreSQL/API uçtan uca otomasyonu ve coverage raporu yok.
Düşük	API contract tutarlılığı	Projects PUT imzası/body, validation/error biçimleri ve dosya adlarında tutarsızlık.
Düşük	Operasyon	Health endpoint, production Swagger koşulu ve rate limiting yok.

29. Gelecek Geliştirme Önerileri

29.1 Yüksek öncelik

Öneri	Fayda	Etkilenen alan	Karmaşıklık
Güvenlik paketlerini güncelle	Bilinen high-severity riskleri azaltır.	Infrastructure csproj ve regression testleri	Düşük-Orta
Secret'ları dışarı taşı	Repository/image içinde anahtar riskini kaldırır.	appsettings, Compose, CI/CD	Orta
Resource authorization ekle	Kullanıcının yalnız yetkili kaynakları değiştirmesini sağlar.	Policy/handler, controller/service	Yüksek
PostgreSQL entegrasyon testleri	Query filter, migration ve constraint'leri gerçek provider ile kanıtlar.	Testcontainers/test project	Orta-Yüksek

29.2 Orta öncelik

- Pagination/filter/sort sözleşmesi ekleyin; repository sorgularını IQueryable/projection veya özel query service ile ölçekleyin.
- Assignment remove/transfer/complete use-case'lerini ve status senkronizasyon kararını tanımlayın.
- Response DTO'larına gerekli Role/RequiredExpertise alanlarını tutarlı map edin; OpenAPI contract testleri ekleyin.
- Refresh token/revocation, parola sıfırlama ve login korumaları ekleyin.
- Health checks ve production'a göre Swagger/Seq yapılandırması ekleyin.

29.3 Opsiyonel/ileri

- Bildirim, aktivite geçmişi, yorum ve dashboard/reporting.
- Task dependency, SLA ve gecikme raporları.
- Optimistic concurrency (row version/xmin) ve idempotency key.
- OpenTelemetry tracing/metrics ve merkezi PII maskeleyme.

30. Sorun Giderme

Belirti	Olası neden	Uygun kontrol/çözüm
Build NU1903	System.Security.Cryptography.Xml 8.0.3 advisory	Paketi uyumlu güvenli sürüme yükseltip tüm testleri çalıştırın; uyarıyı bastırmayın.
DB bağlantısı	Host/port/user/password veya container DNS	Native çalıştırmada localhost; Compose içinde Host=postgres kullanın.
Migration hata	Yanlış startup/project veya hedef DB	İki csproj yolunu ve connection string'i doğrulayın; has-pending-model-changes çalıştırın.
Compose Seq volume	External volume yok	docker volume create taskmanager-seq-data çalıştırın.
Swagger 401	Token yok/geçersiz/expired	Login olun; Authorize alanına yalnız token değerini girin.
Swagger 403	Role claim yetersiz	Admin action için Admin token gerekir; rol güncellendiye yeniden login olun.
400	Validator veya expertise/hierarchy kuralı	field/message array veya ErrorResponse.message değerini inceleyin.
404	Kayıt yok veya soft-deleted	ID'nin entity türünü doğrulayın; TaskTag.Id ile Tag.Id'yi karıştırmayın.
409	Duplicate email/tag/assignment/TaskTag veya cycle	Mevcut aktif kaydı ve composite unique kısıtını kontrol edin.
X-Client-Id	Eski belge/istemci beklentisi	Header'ı kaldırın; güncel API yalnız Bearer JWT kullanır.
Seq görünmüyor	Sink URL/servis/volume	docker compose ps ve logs; API için http://seq:80 override'ını kontrol edin.

31. Sözlük

Terim	Tanım
Entity	Veritabanında kimliği ve yaşam döngüsü olan domain nesnesi.
DTO	API istek/yanıt sözleşmesini entity'den ayıran veri nesnesi.
Repository	Entity veri erişimini soyutlayan arayüz/uygulama.
Unit of Work	Bir use-case değişikliklerini SaveChanges ile kalıcılaştıran sınır.
Middleware	HTTP pipeline'ında cross-cutting işlem yapan bileşen.
Migration	EF modelindeki şema değişikliğinin sürümlenmiş veritabanı adımı.
JWT	Claim içeren imzalı erişim token'ı.
Claim	Token içindeki user ID, email, role gibi doğrulanabilir bilgi.
Role	Kullanıcının yetki düzeyini belirleyen UserRole değeri.
Soft delete	Satırı kaldırmak yerine IsDeleted ile görünmez yapmak.
Audit fields	Oluşturan/değiştiren/silen aktör ve zaman bilgileri.
Many-to-many	TaskAssignment veya TaskTag join entity'siyle kurulan çoktan çoğa ilişki.
Flags enum	Powers-of-two değerleri bitwise OR ile birleştiren enum.
Dependency Injection	Sınıf bağımlılıklarının dışarıdan container ile sağlanması.
FluentValidation	Request DTO kurallarını fluent API ile tanımlayan kütüphane.

32. Nihai Proje Değerlendirmesi

32.1 Güçlü yönler

- Katmanlar ve bağımlılık yönü anlaşılır; service/repository/DTO ayrımı sunum ve onboarding için uygundur.
- Audit ve soft delete merkezi uygulanır; aktif unique indexler veri yaşam döngüsüyle uyumludur.
- Uzmanlık özelliği rol modelini bozmadan [Flags] ile doğru ve genişletilebilir biçimde tasarlanmıştır.
- Exception sözleşmesi, Traceld, Serilog/Seq ve kapsamlı test paketi hata ayıklanabilirliği artırır.
- 131/131 test sonucu staj projesi için iyi bir regresyon güveni sağlar.

32.2 Dengeli olgunluk değerlendirme

Mimari olgunluk eğitim/staj bağlamında orta-iyi düzeydedir. İş kuralları özellikle tag normalizasyonu, duplicate engelleme, görev hiyerarşisi, soft delete ve expertise kontrolünde belirgindir. Test olgunluğu unit seviyesinde güçlü, gerçek PostgreSQL ve HTTP uçtan uca seviyesinde sınırlıdır.

Proje şu haliyle kontrollü development/demo ortamı için uygundur; production-ready olarak değerlendirilmemelidir. Bunun temel nedenleri bilinen güvenlik uyarıları, literal secret riski, geniş mutation yetkileri, refresh/rate-limit eksikleri, gerçek provider entegrasyon testlerinin ve operasyonel health/telemetry mekanizmalarının bulunmamasıdır.

En önemli sonraki adım: Önce güvenlik borcunu kapatın: vulnerable paketi güncelleyin, JWT/DB secret'larını dışarı taşıyın ve kaynak bazlı authorization sınırlarını tanımlayın. Ardından gerçek PostgreSQL entegrasyon testleriyle bu sınırları kanıtlayın.

32.3 Doğrulama durumu

Kontrol	Sonuç
Kaynaklar	Solution/csproj; tüm Domain entity/enum; DTO; interface; service; validator; mapping; controller; middleware; repository; EF configuration; migration; test; appsettings yapısı; Docker dosyaları; README ve doğrulama rehberi incelendi.
Restore	Başarılı; projeler güncel. NU1903 güvenlik uyarıları mevcut.
Build	Başarılı: 0 hata, 10 uyarı çıktısı (5 advisory iki proje üzerinden raporlandı).
Test	131 başarılı, 0 başarısız, 0 atlanan.
EF	Model ile son migration uyumlu; pending model change yok.
Docker	Compose config geçerli; son kontrol anında servisler çalışmıyordu.
Canlı API	Bu dokümantasyon turunda container kapalı olduğundan yeniden çağrılmadı; endpointler controller ve Swagger konfigürasyonundan çapraz doğrulandı.
Coverage	Doğrulanamadı: collector var, yüzde raporu üretilmedi.
Client ID	Projede bulunamadı; güncel davranış yalnız JWT.